

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	T Bhanu Teja	Reg. No.:	192472259
Section / Batch:	2024	Date:	16-07-2026

Code of Conduct

I _Bhanu Teja(192472259)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Bhanu Teja _____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

Algorithm:

1. Import the re module.
2. Read email, password, and mobile number from the user.
3. Validate the email using a regular expression.
4. Validate the password using a regular expression.
5. Validate the mobile number using a regular expression.
6. Display whether each input is valid or invalid.

Program code:

```
import re

def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    if re.fullmatch(pattern, email):
        return "Valid Email"
    return "Invalid Email"

def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$'
    if re.fullmatch(pattern, password):
        return "Strong Password"
    return "Weak Password"

def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    if re.fullmatch(pattern, mobile):
        return "Valid Mobile Number"
    return "Invalid Mobile Number"

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

print("\n----- OUTPUT -----")
print(validate_email(email))
print(validate_password(password))
print(validate_mobile(mobile))
```

Test Case 1 (Valid):

Input:

```
Enter Email: bhanu123@gmail.com
Enter Password: bhanu@123
Enter Mobile Number: 8008131975
```

Processing

- Email matches pattern ✓

- Password satisfies all rules ✓
- Mobile has 10 digits starting with 9 ✓

Output:

```
----- OUTPUT -----  
Valid Email  
Weak Password  
Valid Mobile Number
```

Test Case 2 (Invalid):

Input:

```
Enter Email: 123bhanu@gmail  
Enter Password: bhanu123  
Enter Mobile Number: 5678901234
```

Processing

- Email starts with digit ✗
- Password has no uppercase and special character ✗
- Mobile starts with 5 ✗

Output:

```
----- OUTPUT -----  
Invalid Email  
Weak Password  
Invalid Mobile Number
```

Section 2: Finite State Automata (DFA) Simulator

Algorithm:

1. Define DFA transitions.
2. Read the input string.
3. Start from the initial state.
4. Move to the next state for each character.
5. Display the transition path.
6. If the final state is reached, accept; otherwise reject.

Program Code:

```
def simulate_dfa(string):  
    state = "q0"  
    path = [state]  
  
    transitions = {  
        ("q0", "a"): "q1",  
        ("q0", "b"): "q0",  
        ("q1", "a"): "q1",  
        ("q1", "b"): "q2",  
        ("q2", "a"): "q1",  
        ("q2", "b"): "q0"  
    }  
  
    for ch in string:  
        if (state, ch) in transitions:  
            state = transitions[(state, ch)]  
            path.append(state)  
        else:  
            print("Invalid Input")  
            return  
  
    print("Transition Path:")  
    print(" -> ".join(path))  
  
    if state == "q2":  
        print("Accepted")  
    else:  
        print("Rejected")  
  
text = input("Enter String: ")  
simulate_dfa(text)
```

Test Case 1:

Input & Output :

```
Enter String: abaab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
```

Test Case 2:

Input & Output :

```
Enter String: abba
Transition Path:
q0 -> q1 -> q2 -> q0 -> q1
Rejected
```

Test Case 3:

Input & Output:

```
Enter String: abc
Invalid Input
```

Section 3: Smart Pattern Matching Engine Algorithm

1. Read multiline text.
2. Display menu.
3. Read user choice.
4. Use Regex to search the selected pattern.
5. Display matching results.

Program code : import re

```
text = """
```

```
Meeting on 12/09/2026
```

```
Call 9876543210
```

```
#NLP
```

```
@OpenAI
```

```
natural language processing
```

```
"""
```

```
def search(choice):
```

```
if choice == 1:  
    print(re.findall(r'\d{2}/\d{2}/\d{4}', text))
```

```
elif choice == 2:  
    print(re.findall(r'[6-9]\d{9}', text))
```

```
elif choice == 3:  
    print(re.findall(r'#\w+', text))
```

```
elif choice == 4:  
    print(re.findall(r'@\w+', text))
```

```
elif choice == 5:  
    word = input("Enter Prefix: ")  
    print(re.findall(r'\b' + word + r'\w*', text))
```

```
elif choice == 6:  
    word = input("Enter Suffix: ")  
    print(re.findall(r'\w*' + word + r'\b', text))
```

```
else:  
    print("Invalid Choice")
```

```
print("1.Date")  
print("2.Phone")  
print("3.Hashtag")  
print("4.Mention")  
print("5.Prefix")  
print("6.Suffix")
```

```
choice = int(input("Enter Choice: "))  
search(choice)
```

Input and output of Testcases:

Choice :1

```
Output
1.Date
2.Phone
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 1
['12/09/2026']
```

Choice :2

```
Output
1.Date
2.Phone
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 2
['9876543210']
```

Choice :3

```
Output
1.Date
2.Phone
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 3
['#NLP']
```

Choice: 4

```
Output
1.Date
2.Phone
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 4
['@OpenAI']
```

Choice:5

```
Output
1.Date
2.Phone
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 5
Enter Prefix: nat
['natural']
```

Choice :6

```
Output
1.Date
2.Phone
3.Hashtag
4.Mention
5.Prefix
6.Suffix
Enter Choice: 6
Enter Suffix: ing
['Meeting', 'processing']
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____